CHAPTER – VII

INHERITANCE IN ENCAPSULATION:

- A private variable in a parent class cannot be accessed directly in a child class, even if the child class uses extends.

EXAMPLE:

```
class Example{
        private int n=10;
        public int getN(){
                return n;
        }
}
class Examp1 extends Example{
        void display(){
                System.out.println(getN());
        }
}
```

ADVANTAGES OF ENCAPSULATION:

- Data hiding
- Data security
- Code reusability
- Easy maintenance
- Improved flexibility

ABSTRACTION:

- Abstraction is the process of hiding implementation details and showing only essential detail.

SYNTAX:

```
abstract class className{
    abstract void methodName();
        void normalmethod(){
        }
}
```

EXAMPLE:

```
abstract class Shape {
   abstract void draw();
}
class Circle extends Shape {
   void draw() {
      System.out.println("Drawing Circle");
   }
}
public class AbstractionMain {
   public static void main(String[] args) {
      Circle c = new Circle();
      c.draw();
   }
}
```

ADVANTAGES:

- Data Security
- Code Reusability
- Simplified Maintenance
- Improves Flexibility

POLYMORPHISM:

- Polymorphism means “many forms”
- It allows one object to behave in multiple ways.

TYPES:

- Compile Time – Method Overloading
- Runtime – Method Overriding

COMPILE TIME POLYMORPHISM:

- Multiple methods have the same name but different parameters.

EXAMPLE:

```
class Calculator {
   void add(int a, int b) {
      System.out.println("Sum = " + (a + b));
   }
   void add(int a, int b, int c) {
      System.out.println("Sum = " + (a + b + c));
   }
}
public class CompileTimePoly {
```

```
    public static void main(String[] args) {
        Calculator c = new Calculator();
        c.add(10, 20);
        c.add(10, 20, 30);
    }
}
```

RUNTIME POLYMORPHISM:

- When a subclass provides its own version of a method already defined in the parent class, it is called method overriding.

EXAMPLE:

```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}
class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}
public class RuntimePolyDemo {
    public static void main(String[] args) {
        Animal a = new Dog();
        a.sound();
```

```
    }
}
```